

INTUITION_OS_MASTER_SPEC.md

PART 5

AI SYSTEM DESIGN

LEARNING MEMORY · CONTEXT ENGINE · USER INTELLIGENCE

Version 1.0

Introduction

This section defines the intelligence layer of Intuition OS.

This is the most important part of the system.

The extension can be copied.

The dashboard can be copied.

The prompts can be copied.

The long-term learning memory system cannot be copied easily.

This section defines the system responsible for:

- Personalization
 - Mentorship
 - Growth tracking
 - Reflection generation
 - Recommendation generation
-

AI Architecture Philosophy

The AI should never be the source of truth.

The database is the source of truth.

The AI interprets truth.

The AI does not create truth.

Core Rule

Store facts.

Generate insights.

Never store insights as facts.

Example:

Store:

```text id="m1" Solved Without Hint

Generate:

```text id="m2"  
Increasing Independence

Learning Memory Architecture

The learning memory system stores everything the mentor knows about a user.

Memory Levels

Level 0

Current Interaction

Current mentor conversation.

Very short-lived.

Level 1

Current Session

Current problem.

Current struggle.

Current attempts.

Current hints.

Level 2

Recent Sessions

Last 10–20 sessions.

Used to identify short-term trends.

Level 3

Topic History

Topic-specific learning history.

Examples:

- Arrays
 - Graphs
 - DP
 - Greedy
 - Sliding Window
-

Level 4

Behavior History

Tracks:

- Persistence
 - Optimization
 - Independence
 - Recovery
 - Pattern Recognition
-

Level 5

Long-Term Learning Profile

Months of accumulated evidence.

Most valuable layer.

Context Assembly Engine

The Context Assembly Engine is the most important AI subsystem.

Its job is to construct the right prompt.

Most AI systems fail because they provide too much information or the wrong information.

Context Priority

Priority order:

1. Current Problem
2. Current Session
3. Current User Input
4. Similar Historical Problems
5. Relevant Evidence
6. Learning Profile

Never reverse this order.

Context Assembly Example

User requests help.

Current problem:

Sliding Window Medium.

System retrieves:

Current Session

+

Last Sliding Window sessions

+

Related evidence

+

Current mentor interaction

Prompt generated.

Similar Problem Retrieval

Future enhancement.

Retrieve:

Most similar historical problems.

Based on:

- Tags
- Difficulty
- Topic

Used to create personalized guidance.

User Intelligence Model

The system does not maintain scores.

The system maintains evidence.

Profiles are generated dynamically.

Why Evidence-Based Profiles

Scores become stale.

Evidence remains valid.

Example:

Store:

```
```text id="e1" Solved 6 graph problems without hints.
```

Generate:

```
```text id="e2"  
Graph independence improving.
```

Core Evidence Categories

Independence

Examples:

- Solved without hints
 - Solved without editorial
-

Persistence

Examples:

- Recovered from multiple failures
 - Continued after TLE
-

Optimization

Examples:

- Improved complexity
 - Recovered from TLE
-

Pattern Recognition

Examples:

- Correct pattern identified
 - Correct category selected
-

Recovery

Examples:

- Wrong Answer → Accepted
 - TLE → Accepted
-

Debugging

Examples:

- Fixed implementation errors
 - Corrected edge cases
-

Evidence Confidence

Every evidence item has confidence.

High Confidence

Observed repeatedly.

Medium Confidence

Observed several times.

Low Confidence

Observed once.

Mentor Decision Engine

The mentor should not simply respond.

The mentor should decide.

Every interaction follows:

Observe

↓

Diagnose

↓

Choose Escalation

↓

Respond

Mentor Decision Inputs

Current Problem

Current Session

User Question

Current Escalation Level

Historical Evidence

Mentor Decision Outputs

Observation

Question

Guidance

Escalation Level

Escalation Rules

First Help Request

Default:

Level 1

Reflection.

Repeated Help Request

Level 2

Direction.

Continued Struggle

Level 3

Pattern Hint.

Significant Difficulty

Level 4

Strong Hint.

Explicit Request

Level 5

Solution Path.

Mentor Personalization Rules

Every mentor response should attempt to use history.

Bad:

Use a hash map.

Good:

You solved several hash map problems recently without assistance.

What information would a hash map help you track here?

Reflection Generation Pipeline

Triggered when:

Session ends.

Reflection Inputs

Problem Metadata

Session Metadata

Submission History

Hint History

Editorial History

Mentor Interactions

Evidence

Reflection Outputs

Session Summary

Learning Insight

Behavioral Evidence

Growth Impact

Recommendation

Reflection Example

Session Summary:

You solved this medium problem after one TLE and two submissions.

Learning Insight:

The breakthrough came when you recognized the performance bottleneck.

Behavioral Evidence:

- Solved independently
- Recovered from TLE
- Improved complexity

Growth Impact:

Additional evidence for optimization reasoning.

Recommendation:

Practice another optimization-focused problem.

Reflection Quality Rules

Never invent growth.

Never invent strengths.

Never invent weaknesses.

Every statement must be evidence-backed.

Recommendation Engine

Not part of MVP.

Must be designed now.

Purpose

Answer:

What should the user do next?

Recommendation Inputs

Current Weaknesses

Recent Evidence

Recent Topics

Learning Goals

Recommendation Outputs

Problems

Topics

Exercises

Learning Focus

Example Recommendation

Instead of:

Practice DP.

Generate:

You struggled with state transitions in 3 of your last 5 DP problems.

Recommended:

- House Robber
 - Coin Change
 - Decode Ways
-

User Profile Generation

Profiles are not stored.

Profiles are generated dynamically.

Profile Sections

Current Strengths

Current Weaknesses

Growth Trends

Behavioral Patterns

Recommendations

Strength Generation

Generated from evidence.

Example:

Observed:

Solved 8 graph problems without hints.

Generated:

Graph reasoning appears increasingly independent.

Weakness Generation

Observed:

Viewed editorial in 5 of last 6 DP problems.

Generated:

DP remains dependent on external assistance.

Growth Trends

Generated from time-series evidence.

Examples:

Hint Dependency ↓

Optimization ↑

Pattern Recognition ↑

Weekly Review System

Future feature.

Purpose

Summarize growth.

Weekly Review Sections

Problems Solved

Key Improvements

Recurring Struggles

Recommendations

Monthly Review System

Future feature.

Purpose:

Long-term learning narrative.

Prompt Library

All prompts must be external files.

Never hardcoded.

Prompt Categories

Mentor Prompt

Reflection Prompt

Evidence Prompt

Recommendation Prompt

Profile Prompt

Mentor Prompt Goals

Understand before helping.

Guide before hinting.

Hint before solving.

Reflection Prompt Goals

Teach.

Not congratulate.

Recommendation Prompt Goals

Prioritize learning gaps.

Future AI Evolution

V1

Prompt Engineering

Context Assembly

Evidence Extraction

V2

Topic Memory

Advanced Recommendations

Weekly Reviews

V3

Adaptive Learning Paths

Learning Forecasting

Intervention Prediction

V4

Full Learning Intelligence Layer

Cross-platform learning memory.

LeetCode.

Codeforces.

AtCoder.

System Design.

Frontend.

Backend.

ML.

Unified mentor.

Ultimate Vision

The mentor should eventually understand:

- How the user thinks
- Why the user struggles
- Which interventions help
- What should happen next

better than the user understands themselves.

The system should become a personalized operating system for intellectual growth.

END OF PART 5